GSAP

GreenSock Animation Platform

Complete Masterclass

Beginner ⟶ Intermediate ⟶ Pro

HTML **React + useGSAP** **বাংলা ব্যাখ্যা**

Every Method • Every Plugin • Every Property

useGSAP Hook · Timeline · ScrollTrigger · SplitText · Flip · Draggable · Observer · MotionPath · DrawSVG

CH 01 INTRODUCTION TO GSAP

What is GSAP?

GSAP (GreenSock Animation Platform) is the industry-standard JavaScript animation library trusted by millions of developers worldwide. It lets you animate anything — DOM elements, SVG, Canvas, WebGL, React components, and JavaScript objects — with incredible performance and fine-grained control.

🇧🇩 বাংলা: *GSAP হলো বিশ্বের সবচেয়ে জনপ্রিয় JavaScript অ্যানিমেশন লাইব্রেরি। এটি দিয়ে HTML এলিমেন্ট, SVG, Canvas, WebGL এবং যেকোনো JavaScript অব্জেক্ট অ্যানিমেট করা যায়।*

COMPARE Why GSAP over CSS Animations?

- Up to 20x faster than jQuery; outperforms CSS on complex sequences
- Works in every browser — zero vendor prefixes needed
- Animate ANY numeric property of ANY JavaScript object
- Powerful Timeline control — pause, reverse, scrub, speed up
- 30+ built-in eases plus custom ease support
- Rich plugin ecosystem: ScrollTrigger, SplitText, Draggable, Flip, MorphSVG…

🇧🇩 বাংলা: *CSS অ্যানিমেশনের চেয়ে GSAP অনেক শক্তিশালী — দ্রুততর, সব ব্রাউজারে কাজ করে, এবং যেকোনো প্রপার্টি অ্যানিমেট করতে পারে।*

Installation

HTML CDN (HTML Projects)

⬡ **index.html**

```
<!-- Place before </body> tag -->
<script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.5/gsap.min.js"></script>
<script
src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.5/ScrollTrigger.min.js"></script>
<!-- Plugins auto-register when loaded via CDN — no registerPlugin() needed -->
```

REACT npm (React Projects)

⬡ **Terminal**

```
# Install GSAP core library
npm install gsap

# Install the official React hook
npm install @gsap/react
```

🇧🇩 বাংলা: *React প্রজেক্টে দুটি প্যাকেজ লাগবে: gsap (মূল লাইব্রেরি) এবং @gsap/react (React-এর জন্য useGSAP hook)।*

CH 02 USEGSAP HOOK — THE OFFICIAL REACT WAY

⭐ **IMPORTANT** useGSAP() is the OFFICIAL and CORRECT way to use GSAP in React. Always use this — never useEffect() for GSAP code.

🇧🇩 বাংলা: *useGSAP()* হলো *React*-এ *GSAP* ব্যবহারের সঠিক এবং অফিসিয়াল পদ্ধতি। এটি *useEffect*-এর চেয়ে অনেক সহজ এবং নিরাপদ।

Setup & Registration

```
⬡ main.jsx — one-time setup
// Step 1 — Install (terminal)
// npm install gsap @gsap/react

// Step 2 — Import & Register (do this ONCE in main.jsx or App.jsx)
import gsap from 'gsap';
import { useGSAP } from '@gsap/react';
import { ScrollTrigger } from 'gsap/ScrollTrigger';
import { TextPlugin } from 'gsap/TextPlugin';
import { Flip } from 'gsap/Flip';

// Register the hook + all plugins together
gsap.registerPlugin(useGSAP, ScrollTrigger, TextPlugin, Flip);
// ↑ registerPlugin() prevents React version discrepancies
```

Basic useGSAP Syntax

```
⬡ YourComponent.jsx — basic pattern
import { useRef } from 'react';
import gsap from 'gsap';
import { useGSAP } from '@gsap/react';

gsap.registerPlugin(useGSAP);

export default function Box() {
  const container = useRef();

  useGSAP(() => {
    // ✅ All GSAP code lives here
    // String selectors are scoped to <container> automatically
    // Animations are AUTOMATICALLY reverted when component unmounts!
    gsap.to('.box', { x: 360, duration: 1, ease: 'power2.out' });

  }, { scope: container }); // scope = optional but always recommended

  return (
    <div ref={container}>
      <div className="box" style={{ width: 80, height: 80, background: '#0AE448' }} />
    </div>
```

```
  );
}
```

🇧🇩 বাংলা: *useGSAP*-এর ভেতরে লেখা সব অ্যানিমেশন *component unmount* হলে স্বয়ংক্রিয়ভাবে পরিষ্কার হয়ে যায়। আলাদা *cleanup* লেখার দরকার নেই।

OLD vs NEW — Side by Side

❌ OLD WAY *useEffect + gsap.context()*	✅ NEW WAY *useGSAP — Official Hook*
`useEffect(() => { const ctx = gsap.context(() => { gsap.to('.box', { x: 360 }); }, container); return () => ctx.revert();}, []);`	`useGSAP(() => { gsap.to('.box', { x: 360 }); // auto-cleanup! ✓}, { scope: container });`

🇧🇩 বাংলা: পার্থক্য: *useEffect*-এ নিজে *ctx.revert()* লিখতে হতো। *useGSAP*-এ সেটা অটোমেটিক হয়। কম কোড, কম বাগ, বেশি শান্তি।

All useGSAP Options

Method / Property	Description / Example
`scope`	Ref to scope string selectors inside. Prevents affecting other components.
`dependencies`	Array like useEffect deps. Re-runs the hook when values change.
`revertOnUpdate`	true (default) = revert animations before re-running. false = don't revert.

```
⬡ useGSAP — all options
useGSAP(
  () => {
    gsap.from('.card', { y: 60, opacity: 0, stagger: 0.1 });
  },
  {
    scope:          container,     // limit selector scope to this element
    dependencies:   [isOpen],      // re-run when isOpen changes
    revertOnUpdate: true,          // revert prev animations before re-running
  }
);
```

contextSafe() — Safe Event Handlers

ℹ **INFO** contextSafe() wraps callbacks (onClick, onMouseMove, setTimeout) so their GSAP code is included in automatic cleanup.

```
⬡ contextSafe — event handlers
import { useRef } from 'react';
import gsap from 'gsap';
import { useGSAP } from '@gsap/react';
```

```
gsap.registerPlugin(useGSAP);

export default function InteractiveBox() {
  const container = useRef();

  // Destructure contextSafe from useGSAP return value
  const { contextSafe } = useGSAP({ scope: container });

  // ✅ Wrap ALL event handlers with contextSafe
  const handleClick = contextSafe(() => {
    gsap.to('.box', { rotation: '+=360', duration: 0.6, ease: 'back.out(2)' });
  });

  const handleHover = contextSafe(() => {
    gsap.to('.box', { scale: 1.15, duration: 0.25, ease: 'power2.out' });
  });

  const handleLeave = contextSafe(() => {
    gsap.to('.box', { scale: 1, duration: 0.4, ease: 'elastic.out(1, 0.4)' });
  });

  return (
    <div ref={container}>
      <div
        className="box"
        onClick={handleClick}
        onMouseEnter={handleHover}
        onMouseLeave={handleLeave}
      />
    </div>
  );
}
```

🇧🇩 বাংলা: *contextSafe() ছাড়া event handler-এর GSAP কোড cleanup-এর বাইরে থাকে এবং memory leak হতে পারে। সবসময় contextSafe() ব্যবহার করুন।*

useGSAP with Timeline + ScrollTrigger

```
⬡ Full hero section — timeline + ScrollTrigger
import { useRef } from 'react';
import gsap from 'gsap';
import { useGSAP } from '@gsap/react';
import { ScrollTrigger } from 'gsap/ScrollTrigger';

gsap.registerPlugin(useGSAP, ScrollTrigger);

export default function HeroSection() {
  const container = useRef();

  useGSAP(() => {
    // ── Timeline ──────────────────────────────────────────
```

```
      const tl = gsap.timeline({
        defaults: { ease: 'power3.out', duration: 0.75 }
      });

      tl.from('.nav',       { y: -50, opacity: 0 })
        .from('.hero-h1',   { y: 80, opacity: 0 }, '-=0.3')
        .from('.hero-sub',  { y: 40, opacity: 0 }, '-=0.4')
        .from('.hero-cta',  { scale: 0.8, opacity: 0 }, '-=0.3');

      // ── ScrollTrigger — auto-cleaned up with component! ──────
      gsap.from('.feature-card', {
        y: 70, opacity: 0,
        stagger: 0.15, duration: 0.8,
        scrollTrigger: {
          trigger: '.features',
          start: 'top 78%',
          toggleActions: 'play none none reverse',
        }
      });

    }, { scope: container }); // ← everything scoped + auto-cleanup

    return (
      <div ref={container}>
        <nav className="nav">...</nav>
        <h1 className="hero-h1">Amazing Product</h1>
        <p className="hero-sub">Crafted with GSAP</p>
        <button className="hero-cta">Get Started</button>
        <div className="features">
          {['Fast','Smooth','Powerful'].map((f, i) => (
            <div key={i} className="feature-card">{f}</div>
          ))}
        </div>
      </div>
    );
  }
```

CH 03 **CORE ANIMATION METHODS**

The 4 Essential Methods

Method / Property	Description / Example
`gsap.to()`	Animate FROM current state TO specified values. Most common.
`gsap.from()`	Animate FROM specified values TO current state. Entrance animations.
`gsap.fromTo()`	Explicitly define BOTH start and end values. Maximum control.
`gsap.set()`	Instantly set properties with NO animation (duration 0).

TO gsap.to() — Animate TO

🇧🇩 বাংলা: *gsap.to() মানে:* এলিমেন্ট বর্তমান অবস্থান থেকে নির্দিষ্ট মানে যাবে। সবচেয়ে বেশি ব্যবহৃত।

⬡ **gsap.to()**

```
// HTML
gsap.to('.box', {
  x: 300, opacity: 0.5, scale: 1.2,
  duration: 2, ease: 'power2.out',
});

// React (useGSAP)
useGSAP(() => {
  gsap.to('.box', { x: 300, opacity: 0.5, scale: 1.2, duration: 2, ease: 'power2.out'
});
}, { scope: container });
```

FROM gsap.from() — Animate FROM

🇧🇩 বাংলা: *gsap.from() মানে:* আপনি যে মান দেবেন সেখান থেকে বর্তমান অবস্থানে আসবে। *Entrance animation-*এর জন্য।

⬡ **gsap.from()**

```
// HTML
gsap.from('.cards', {
  y: 80, opacity: 0, scale: 0.85, stagger: 0.15,
  duration: 0.9, ease: 'power3.out',
});

// React (useGSAP)
useGSAP(() => {
  gsap.from('.card', {
    y: 80, opacity: 0, scale: 0.85,
    stagger: 0.15, duration: 0.9, ease: 'power3.out',
  });
}, { scope: container });
```

FROM→TO gsap.fromTo() — Full Control

🇧🇩 বাংলা: *gsap.fromTo()* দিয়ে শুরু এবং শেষ দুটি অবস্থাই নির্ধারণ করেন। সর্বোচ্চ কন্ট্রোল।

```
⬡ gsap.fromTo()
// Both HTML and React — syntax identical
gsap.fromTo(
  '.hero-title',
  { y: -60, opacity: 0, rotation: -5 },   // FROM values
  { y: 0, opacity: 1, rotation: 0,         // TO values
    duration: 1.2, ease: 'power2.out' }
);
```

SET gsap.set() — Instant Set

🇧🇩 বাংলা: *gsap.set()* কোনো অ্যানিমেশন ছাড়াই তাৎক্ষণিকভাবে প্রপার্টি সেট করে।

```
⬡ gsap.set()
// Set initial state instantly
gsap.set('.box', { x: 0, opacity: 0, scale: 0.9 });

// Clear all GSAP inline styles
gsap.set('.box', { clearProps: 'all' });
```

CH 04 ANIMATABLE PROPERTIES — COMPLETE REFERENCE

💡 **TIP** Always use x/y/scale/rotation for movement — they run on the GPU and never cause browser layout recalculation.

🇧🇩 বাংলা: পারফরমেন্সের জন্য সবসময় *x, y, scale, rotation* ব্যবহার করুন। *left/top* ব্যবহার করলে *browser reflow* হয় যা অ্যানিমেশন ধীর করে।

Transform Properties (GPU Accelerated ⚡)

```
⬡ Transform Properties
gsap.to('.el', {
  // POSITION — GPU accelerated ✅
  x: 200,            y: -100,           z: 50,
  xPercent: 50,      yPercent: -25,

  // ROTATION
  rotation: 360,     rotationX: 180,    rotationY: 90,
  skewX: 20,         skewY: 10,

  // SCALE
  scale: 2,          scaleX: 1.5,       scaleY: 0.8,

  // TRANSFORM ORIGIN (pivot point)
  transformOrigin: 'center center',   // default
  transformOrigin: 'top left',
  transformOrigin: '0% 100%',         // bottom left

  duration: 1
});
```

CSS & Visual Properties

```
⬡ CSS Properties
gsap.to('.el', {
  // OPACITY
  opacity: 0.5,
  autoAlpha: 0,    // GSAP special: opacity 0 + visibility:hidden combined

  // COLORS
  backgroundColor: '#1DB954',
  color: 'blue',
  borderColor: '#333',

  // DIMENSIONS (avoid — causes reflow ⚠)
  width: 200,        height: 100,
  padding: 20,       borderRadius: 50,

  // TEXT
```

```
    fontSize: '2rem',  lineHeight: 1.8,   letterSpacing: '0.1em',

    duration: 1
  });
```

🇧🇩 বাংলা: *autoAlpha একটি বিশেষ GSAP প্রপার্টি। opacity: 0 করলে visibility: hidden ও সেট করে, যা accessibility-র জন্য ভালো।*

Special GSAP Control Properties

Method / Property	Description / Example
`duration: 2`	Animation length in seconds
`delay: 0.5`	Wait 0.5s before starting
`repeat: -1`	Repeat forever. repeat: 3 = plays 4 times total
`yoyo: true`	Reverse direction on alternate repeats
`stagger: 0.1`	0.1s delay between each element (for arrays)
`ease: 'power2.out'`	Easing function (see Chapter 5)
`overwrite: 'auto'`	Kill conflicting tweens automatically
`paused: true`	Create but don't play — control manually
`id: 'myAnim'`	Name this tween for later reference
`onStart: () => {}`	Callback when animation starts
`onUpdate: () => {}`	Callback every frame while active
`onComplete: () => {}`	Callback when animation finishes
`keyframes: [...]`	Multiple keyframe states in one tween

CH 05 EASING — THE SOUL OF ANIMATION

ℹ **INFO** Format: 'easeName.type' · .in = slow start · .out = slow end (most natural) · .inOut = slow both

🇧🇩 বাংলা: *.out* সবচেয়ে বেশি ব্যবহৃত হয় কারণ এটি প্রকৃতির মতো দেখায় — দ্রুত শুরু হয়ে ধীরে থামে।

All Built-in Eases

⬡ **All Built-in Eases**

```
// ── POWER (most common) ──────────────────────────
'none'            // linear — no easing at all
'power1.out'      // gentle deceleration
'power2.out'      // ⭐ MOST USED — feels natural & professional
'power3.out'      // snappier deceleration
'power4.out'      // very fast start, gradual stop

// ── BACK (overshoot) ─────────────────────────────
'back.out(1.7)'   // ⭐ slight overshoot at end — feels bouncy
'back.out(3)'     // exaggerated overshoot
'back.in(1.7)'    // overshoot at start

// ── ELASTIC (springy) ────────────────────────────
'elastic.out(1, 0.3)'  // ⭐ spring/rubber-band effect
'elastic.in'
'elastic.inOut'

// ── BOUNCE (ball) ────────────────────────────────
'bounce.out'      // bounces at end like a dropped ball
'bounce.in'       // bounces at start

// ── OTHERS ───────────────────────────────────────
'expo.out'        // very sharp then very slow
'circ.out'        // circular arc
'sine.inOut'      // very smooth sine wave
'steps(8)'        // 8 discrete jumps (like CSS steps())
'slow(0.7,0.7)'   // slow in the middle, fast at edges
```

CustomEase Plugin

⬡ **CustomEase**

```
import { CustomEase } from 'gsap/CustomEase';
gsap.registerPlugin(useGSAP, CustomEase);

// Create ONCE outside the component
CustomEase.create('myEase', 'M0,0 C0.14,0 0.242,0.438 0.272,0.561...');

// Use anywhere
useGSAP(() => {
  gsap.to('.box', { x: 400, ease: 'myEase', duration: 1.5 });
```

```
}, { scope: container });

// 🔗 Build custom eases visually: https://gsap.com/docs/v3/Eases
```

CH 06 TIMELINE — SEQUENCE & ORCHESTRATE

⭐ **IMPORTANT** Timelines are GSAP's most powerful feature. Chain animations, control the whole sequence as one unit, and use the position parameter for complex choreography.

🇧🇩 বাংলা: *Timeline* দিয়ে একাধিক অ্যানিমেশন সিকোয়েন্স করা যায়। *delay* গণনার ঝামেলা নেই।

Basic Timeline

⬡ **React — Timeline**

```
// React — Timeline inside useGSAP (correct way)
useGSAP(() => {
  const tl = gsap.timeline({
    defaults: { ease: 'power3.out', duration: 0.75 },  // shared defaults
    onComplete: () => console.log('All done!'),
    repeat: 0,
    paused: false,
  });

  tl.from('.nav',       { y: -50, opacity: 0 })
    .from('.hero-h1',   { y: 80,  opacity: 0 }, '-=0.3')  // overlap 0.3s
    .from('.hero-sub',  { y: 40,  opacity: 0 }, '-=0.4')
    .from('.hero-cta',  { scale: 0.8, opacity: 0 }, '-=0.3')
    .from('.card', { y: 50, opacity: 0, stagger: 0.12 }, '+=0.1');

}, { scope: container });
```

Position Parameter

ℹ **INFO** The 3rd argument to timeline methods — controls WHEN each animation starts relative to the timeline.

⬡ **Position Parameter**

```
const tl = gsap.timeline();

tl.to('.a', { x: 100, duration: 1 })
  // Absolute (seconds from timeline start)
  .to('.b', { x: 100, duration: 1 }, 0)       // starts at 0s (simultaneous)
  .to('.c', { x: 100, duration: 1 }, 0.5)     // starts at exactly 0.5s
  // Relative (from end of previous tween)
  .to('.d', { x: 100, duration: 1 }, '-=0.5') // 0.5s BEFORE prev ends ← overlap
  .to('.e', { x: 100, duration: 1 }, '+=0.3') // 0.3s AFTER prev ends ← gap
  // Labels
  .addLabel('phase2', 1.5)
  .to('.f', { x: 100 }, 'phase2')             // at label
  .to('.g', { x: 100 }, 'phase2+=0.2')        // 0.2s after label
  // Align with previous
  .to('.h', { x: 100 }, '<')                  // same START as previous
```

```
    .to('.i', { x: 100 }, '<50%')              // 50% into previous duration
    .to('.j', { x: 100 }, '>')                 // when previous ENDS
```

Timeline Control Methods

Method / Property	Description / Example
`tl.play()`	Play from current position
`tl.pause()`	Pause at current position
`tl.resume()`	Resume after pause
`tl.reverse()`	Play backwards from current position
`tl.restart()`	Jump to start and play forward
`tl.seek(1.5)`	Jump to 1.5 seconds
`tl.seek('label')`	Jump to a named label
`tl.progress(0.5)`	Jump to 50% of total duration
`tl.timeScale(2)`	Play at 2× speed. -1 = backwards
`tl.kill()`	Destroy the timeline completely
`tl.clear()`	Remove all tweens, keep timeline object
`tl.duration()`	Get total duration in seconds
`tl.isActive()`	Is it currently animating?

Paused Timeline + contextSafe (React)

```
⬡ Paused Timeline + contextSafe
export default function AnimatedMenu() {
  const container = useRef();
  const tlRef = useRef(null);

  const { contextSafe } = useGSAP(() => {
    // Create paused timeline and store in ref
    tlRef.current = gsap.timeline({ paused: true })
      .to('.menu', { x: 0, duration: 0.5, ease: 'power2.out' })
      .from('.menu-item', { opacity: 0, x: -20, stagger: 0.08 });
  }, { scope: container });

  // Wrap click handlers with contextSafe!
  const open  = contextSafe(() => tlRef.current?.play());
  const close = contextSafe(() => tlRef.current?.reverse());

  return (
    <div ref={container}>
      <button onClick={open}>Open Menu</button>
      <button onClick={close}>Close Menu</button>
      <nav className="menu" style={{ transform: 'translateX(-300px)' }}>
        <div className="menu-item">Home</div>
```

```
          <div className="menu-item">About</div>
          <div className="menu-item">Work</div>
        </nav>
      </div>
    );
  }
```

CH 07 UTILITY METHODS & GSAP.UTILS

Core Utility Methods

Method / Property	Description / Example
`gsap.getProperty(el, prop)`	Get current computed value: gsap.getProperty('.box', 'x')
`gsap.quickTo(el, prop, vars)`	Create a fast update function. Best for mouse tracking.
`gsap.quickSetter(el, prop)`	Create an instant (no-tween) setter. Fastest possible.
`gsap.getById('myAnim')`	Retrieve a tween/timeline by its id
`gsap.getTweensOf(target)`	Get array of all active tweens on target
`gsap.killTweensOf(target)`	Kill all tweens on target (optional: specific props)
`gsap.registerPlugin(...)`	Register plugins before use
`gsap.matchMedia()`	Responsive animations per breakpoint
`gsap.context(fn, scope)`	Scoped context (older pattern, use useGSAP instead)

PERF gsap.quickTo() — Mouse Tracking

🇧🇩 বাংলা: *gsap.quickTo() cursor tracking-এর জন্য সেরা। প্রতিটি mousemove-এ নতুন tween তৈরি না করে সরাসরি target update করে।*

```
⬡ React — quickTo cursor
// React — custom cursor with useGSAP
export default function CustomCursor() {
  const cursorRef = useRef(null);
  const xToRef = useRef(null);
  const yToRef = useRef(null);

  useGSAP(() => {
    xToRef.current = gsap.quickTo(cursorRef.current, 'x', { duration: 0.4, ease:
'power3' });
    yToRef.current = gsap.quickTo(cursorRef.current, 'y', { duration: 0.4, ease:
'power3' });

    const onMove = (e) => {
      xToRef.current(e.clientX);
      yToRef.current(e.clientY);
    };
    window.addEventListener('mousemove', onMove);
    return () => window.removeEventListener('mousemove', onMove); // cleanup!
  });

  return (
    <div ref={cursorRef} style={{
      position:'fixed', width:20, height:20, borderRadius:'50%',
      background:'white', pointerEvents:'none', zIndex:9999,
      transform:'translate(-50%,-50%)'
```

```
      }} />
    );
  }
```

gsap.utils — Complete Reference

Method / Property	Description / Example
`clamp(min, max, val)`	Clamp value between min and max
`mapRange(iMin,iMax,oMin,oMax)`	Map a value from one range to another
`normalize(min, max, val)`	Normalize to 0–1 range
`interpolate(start, end, t)`	Lerp between two values (colors too!)
`snap(increment, val)`	Snap to nearest multiple or array value
`random(min, max)`	Get a random number. Pass true to get a function.
`shuffle(array)`	Shuffle array in-place
`wrap(min, max, val)`	Wrap value within range (great for carousels)
`distribute({base,amount,...})`	Distribute values across elements by position
`toArray('.selector')`	Convert NodeList / selector string to real Array
`pipe(...fns)`	Chain multiple functions together

⬡ **gsap.utils examples**

```
// Quick examples
gsap.utils.clamp(0, 100, 150);          // → 100
gsap.utils.mapRange(0, 100, 0, 360, 50); // → 180
gsap.utils.normalize(0, 100, 75);       // → 0.75
gsap.utils.interpolate('#f00', '#00f', 0.5); // → purple
gsap.utils.snap(50, 74);                // → 50
gsap.utils.random(0, 100);              // → e.g. 42.7
gsap.utils.wrap(0, 5, 7);               // → 2
gsap.utils.toArray('.card');            // → real Array

// Pipe: clamp → map → round
const xTransform = gsap.utils.pipe(
  gsap.utils.clamp(0, 100),
  gsap.utils.mapRange(0, 100, 0, 360),
  Math.round
);
xTransform(150); // → 360
```

CH 08 SCROLLTRIGGER PLUGIN

🆕 **NEW** ScrollTrigger is the #1 most used GSAP plugin. It ties any GSAP animation to scroll position.

🇧🇩 বাংলা: *ScrollTrigger GSAP-এর সবচেয়ে জনপ্রিয় plugin। Scroll করলে অ্যানিমেশন ট্রিগার হয়, parallax effect, pin, horizontal scroll সব করা যায়।*

Basic Setup (React)

⬡ **React — ScrollTrigger basics**

```
import { ScrollTrigger } from 'gsap/ScrollTrigger';
gsap.registerPlugin(useGSAP, ScrollTrigger);

useGSAP(() => {
  // Fade-in elements on scroll
  gsap.from('.scroll-item', {
    y: 60, opacity: 0,
    stagger: 0.15, duration: 0.8, ease: 'power3.out',
    scrollTrigger: {
      trigger: '.section',  // element that triggers animation
      start: 'top 80%',     // [trigger edge] [viewport edge]
      toggleActions: 'play none none reverse',
      markers: true,        // ← debug markers (REMOVE in production!)
    }
  });

  // Scrub — animation directly linked to scroll position
  gsap.to('.parallax', {
    y: -200, ease: 'none',
    scrollTrigger: {
      trigger: '.hero', start: 'top top', end: 'bottom top',
      scrub: 1,  // 1s smooth lag behind scroll
    }
  });

}, { scope: container }); // ScrollTrigger is AUTO-CLEANED on unmount!
```

🇧🇩 বাংলা: *useGSAP-এর ভেতরে ScrollTrigger ব্যবহার করলে component unmount হলে সব ScrollTrigger instance স্বয়ংক্রিয়ভাবে destroy হয়।*

start / end Positions

Method / Property	Description / Example
`"top 80%"`	Trigger's top edge hits 80% down the viewport
`"top center"`	Trigger's top hits viewport center
`"center center"`	Trigger's center hits viewport center
`"bottom top"`	Trigger's bottom hits top of viewport

Method / Property	Description / Example
`"top+=100 bottom"`	Trigger top +100px hits viewport bottom
`"+=500"`	End 500px after start position

All ScrollTrigger Properties

Method / Property	Description / Example
`trigger`	Element that triggers the animation
`start / end`	'[triggerEdge] [viewportEdge]' — when to start/stop
`scrub: 1`	Link animation to scroll. Number = smoothing lag in seconds
`pin: true`	Stick the trigger element while scrolling through
`snap`	Snap scroll to keyframes: snap: 0.25 or snap: {snapTo: 'labels'}
`toggleActions`	'play none none reverse' — 4 actions: enter, leave, enterBack, leaveBack
`toggleClass`	Add/remove a CSS class as element enters/leaves
`markers: true`	Show debug start/end markers (remove in production!)
`onEnter`	Callback when scrolled into view
`onLeave`	Callback when scrolled past
`onUpdate`	Runs every frame: (self) => self.progress
`invalidateOnRefresh`	Recalculate on window resize

ScrollTrigger.batch() — Many Elements

⬡ **ScrollTrigger.batch()**

```
// Efficiently handle MANY scroll-triggered elements
ScrollTrigger.batch('.card', {
  onEnter: (elements) => {
    gsap.from(elements, { opacity: 0, y: 60, stagger: 0.1, overwrite: true });
  },
  onLeave: (elements) => {
    gsap.set(elements, { opacity: 0, overwrite: true });
  },
  onEnterBack: (elements) => {
    gsap.to(elements, { opacity: 1, y: 0, stagger: 0.1, overwrite: true });
  },
  start: 'top 85%',
  end: 'bottom 15%',
});
```

Horizontal Scroll Section (React)

⬡ **Horizontal Scroll**

```
useGSAP(() => {
  const panels = gsap.utils.toArray('.panel');

  gsap.to(panels, {
    xPercent: -100 * (panels.length - 1),
    ease: 'none',
    scrollTrigger: {
      trigger: '.h-wrapper',
      pin: true,
      scrub: 1,
      snap: 1 / (panels.length - 1),
      end: () => '+=' + (panels.length - 1) * window.innerWidth,
    }
  });
}, { scope: container });
```

CH 09 ALL GSAP PLUGINS

Plugin Overview

Method / Property	Description / Example
`ScrollTrigger`	Scroll-based animations, pin, scrub, parallax — Free
`TextPlugin`	Typewriter / text reveal effects — Free
`DrawSVGPlugin`	Animate SVG path drawing — Club GSAP
`MorphSVGPlugin`	Morph one SVG shape into another — Club GSAP
`MotionPathPlugin`	Move elements along an SVG path — Free
`SplitText`	Split text into chars/words/lines for animation — Club GSAP
`Flip`	FLIP layout transition animations — Free
`Draggable`	Make any element draggable/throwable — Free
`Observer`	Unified scroll/touch/pointer event handler — Free
`ScrollSmoother`	Smooth scrolling + parallax effects — Club GSAP
`CustomEase`	Create custom cubic-bezier eases — Free
`CustomBounce`	Custom bounce ease with squash — Club GSAP
`CustomWiggle`	Wiggle/jiggle animation ease — Club GSAP

TextPlugin — Typewriter Effect

🇧🇩 বাংলা: *TextPlugin* দিয়ে টাইপরাইটার ইফেক্ট তৈরি করা যায় — টেক্সট একটা একটা করে টাইপ হয়।

⬡ **TextPlugin**

```
import { TextPlugin } from 'gsap/TextPlugin';
gsap.registerPlugin(useGSAP, TextPlugin);

useGSAP(() => {
  // Character-by-character typewriter
  gsap.to('.typewriter', {
    duration: 2, text: 'Hello, World!', ease: 'none',
  });

  // Word-by-word reveal
  gsap.to('.words', {
    duration: 3, text: { value: 'Word by word', delimiter: ' ' }, ease: 'none',
  });
}, { scope: container });
```

SplitText Plugin — Per-Character Animation

🇧🇩 বাংলা: *SplitText* দিয়ে টেক্সটকে অক্ষর, শব্দ বা লাইনে ভাগ করে প্রতিটি আলাদাভাবে অ্যানিমেট করা যায়।

```
⬡ SplitText
import { SplitText } from 'gsap/SplitText';
gsap.registerPlugin(useGSAP, SplitText);

useGSAP(() => {
  const split = new SplitText('.headline', {
    type: 'chars,words,lines',
  });

  gsap.from(split.chars, {
    opacity: 0, y: 50, rotationX: -90,
    stagger: 0.02, duration: 0.8,
    ease: 'back.out(2)',
    transformOrigin: '0% 50% -50',
  });

  // split.chars / split.words / split.lines
  // Returns arrays of DOM elements — animate them however you want!
}, { scope: container });
```

Flip Plugin — Layout Transitions

🇧🇩 বাংলা: *Flip plugin দিয়ে layout পরিবর্তনকে সুন্দরভাবে অ্যানিমেট করা যায়। Grid থেকে List view switching।*

```
⬡ Flip Plugin
import { Flip } from 'gsap/Flip';
gsap.registerPlugin(useGSAP, Flip);

export default function FlipDemo() {
  const container = useRef();
  const [grid, setGrid] = useState(true);

  const { contextSafe } = useGSAP({ scope: container });

  const toggle = contextSafe(() => {
    const state = Flip.getState('.card'); // 1. capture
    setGrid(v => !v);                     // 2. change DOM

    // 3. animate — call right after state change
    Flip.from(state, {
      duration: 0.6, ease: 'power2.inOut',
      stagger: 0.05, absolute: true,
    });
  });

  return (
    <div ref={container}>
      <button onClick={toggle}>Toggle Layout</button>
      <div className={grid ? 'grid' : 'list'}>
        {[1,2,3,4,5,6].map(i => <div key={i} className="card">Card {i}</div>)}
      </div>
    </div>
  );
```

```
}
```

Draggable Plugin

🇧🇩 বাংলা: *Draggable* দিয়ে যেকোনো এলিমেন্টকে *drag* করার যোগ্য করা যায়।

```
⬡ Draggable
import { Draggable } from 'gsap/Draggable';
gsap.registerPlugin(useGSAP, Draggable);

useGSAP(() => {
  Draggable.create('.card', {
    type: 'x,y',
    bounds: container.current,
    snap: {
      x: (v) => Math.round(v / 50) * 50,
      y: (v) => Math.round(v / 50) * 50,
    },
    onDragStart: () => gsap.to('.card', { scale: 1.1, duration: 0.2 }),
    onDragEnd:   () => gsap.to('.card', { scale: 1,   duration: 0.3 }),
    cursor: 'grab', activeCursor: 'grabbing',
  });
}, { scope: container });
```

Observer Plugin

```
⬡ Observer — fullpage scroll
import { Observer } from 'gsap/Observer';
gsap.registerPlugin(useGSAP, Observer);

useGSAP(() => {
  let current = 0;
  const sections = gsap.utils.toArray('.section');

  Observer.create({
    type: 'wheel,touch',
    onUp:   () => { if (current > 0) goTo(--current) },
    onDown: () => { if (current < sections.length-1) goTo(++current) },
    tolerance: 10,
    preventDefault: true,
  });

  function goTo(index) {
    gsap.to(window, { scrollTo: sections[index], duration: 0.8, ease: 'power2.inOut' });
  }
});
```

gsap.matchMedia() — Responsive Animations

🇧🇩 বাংলা: *gsap.matchMedia() দিয়ে বিভিন্ন screen size এবং prefers-reduced-motion-এর জন্য আলাদা অ্যানিমেশন তৈরি করা যায়।*

⬡ **gsap.matchMedia()**

```
useGSAP(() => {
  const mm = gsap.matchMedia();

  mm.add('(min-width: 800px)', () => {
    gsap.from('.hero-text',  { x: -200, opacity: 0, duration: 1 });
    gsap.from('.hero-image', { x:  200, opacity: 0, duration: 1 });
  });

  mm.add('(max-width: 799px)', () => {
    gsap.from('.hero-text',  { y: -60, opacity: 0, duration: 0.8 });
  });

  // Multiple conditions at once
  mm.add({
    isDesktop:    '(min-width: 900px)',
    isMobile:     '(max-width: 899px)',
    noMotion:     '(prefers-reduced-motion: reduce)',
  }, (ctx) => {
    const { isDesktop, noMotion } = ctx.conditions;
    if (noMotion) return; // skip all animations
    if (isDesktop) { /* desktop animations */ }
  });

}, { scope: container });
```

Function-Based Values

ℹ **INFO** Any GSAP value can be a function: (index, target, targets) => value. Each element gets a unique computed value.

⬡ **Function-Based Values**

```
useGSAP(() => {
  gsap.to('.item', {
    x:       () => gsap.utils.random(-200, 200),   // random per element
    y:       (i) => i * 60,                        // based on index
    scale:   (i, target) => parseFloat(target.dataset.scale) || 1,
    opacity: (i, t, all) => 1 - i / all.length * 0.5,
    duration: (i) => 0.4 + i * 0.08,
    stagger: 0.04,
  });
}, { scope: container });
```

gsap.ticker — Every Frame

🇧🇩 বাংলা: *gsap.ticker হলো GSAP-এর requestAnimationFrame লুপ। প্রতিটি frame-এ কোড চালানো যায় — Three.js, Canvas-এর জন্য উপযুক্ত।*

⬡ **gsap.ticker**

```
useGSAP(() => {
  const fn = (time, deltaTime, frame) => {
    // time = total elapsed seconds
    // deltaTime = ms since last frame
    mesh.rotation.y += deltaTime * 0.001; // Three.js mesh
  };
  gsap.ticker.add(fn);
  return () => gsap.ticker.remove(fn); // cleanup returned from useGSAP
});

// Global settings
gsap.ticker.lagSmoothing(500, 33); // prevent jump after tab switch
gsap.globalTimeline.timeScale(0.5); // slow ALL animations to half speed
```

Magnetic Button — Pro Effect

⬡ **Magnetic Button**

```
export default function MagneticButton({ children }) {
  const btnRef = useRef();
  const { contextSafe } = useGSAP({ scope: btnRef });

  const onMove = contextSafe((e) => {
    const rect = btnRef.current.getBoundingClientRect();
    const x = (e.clientX - rect.left - rect.width  / 2) * 0.35;
    const y = (e.clientY - rect.top  - rect.height / 2) * 0.35;
    gsap.to(btnRef.current, { x, y, duration: 0.3, ease: 'power2.out' });
  });

  const onLeave = contextSafe(() => {
    gsap.to(btnRef.current, { x: 0, y: 0, duration: 0.6, ease: 'elastic.out(1, 0.4)' });
  });

  return (
    <button ref={btnRef} onMouseMove={onMove} onMouseLeave={onLeave}>
      {children}
    </button>
  );
}
```

Animated Counter with ScrollTrigger

⬡ **Animated Counter**

```
useGSAP(() => {
```

```
  document.querySelectorAll('[data-count]').forEach(el => {
    const target = parseFloat(el.dataset.count);
    const isDecimal = target % 1 !== 0;
    const obj = { val: 0 };

    gsap.to(obj, {
      val: target,
      duration: 2.5,
      ease: 'power2.out',
      onUpdate: () => {
        el.textContent = isDecimal
          ? obj.val.toFixed(1) + '%'
          : Math.round(obj.val).toLocaleString();
      },
      scrollTrigger: { trigger: el, start: 'top 80%' }
    });
  });
}, { scope: container });

// HTML usage:
// <span data-count="9847"></span>  → animates to 9,847
// <span data-count="99.9"></span>  → animates to 99.9%
```

Page Loader Animation

```
⬡ Page Loader
export default function Loader({ onDone }) {
  const ref = useRef();

  useGSAP(() => {
    const tl = gsap.timeline({
      onComplete: () => gsap.to(ref.current, {
        opacity: 0, duration: 0.5,
        pointerEvents: 'none', onComplete: onDone,
      })
    });

    tl.from('.loader-icon', { scale: 0, duration: 0.6, ease: 'back.out(2)' })
      .from('.loader-bar',  { scaleX: 0, transformOrigin: 'left',
                              duration: 1.5, ease: 'power2.inOut' }, '-=0.2')
      .from('.loader-text', { opacity: 0, y: 20, duration: 0.4 }, '-=0.5');
  }, { scope: ref });

  return (
    <div ref={ref} style={{ position:'fixed', inset:0, background:'#0D1117',
      display:'flex', flexDirection:'column', alignItems:'center',
justifyContent:'center' }}>
      <div className="loader-icon" style={{ fontSize:64, color:'#0AE448' }}>◆</div>
      <div style={{ width:260, height:3, background:'#1E293B', marginTop:24 }}>
        <div className="loader-bar" style={{ height:'100%', background:'#0AE448',
scaleX:0 }} />
      </div>
```

```
        <p className="loader-text" style={{ color:'#94A3B8', marginTop:12
}}>Loading...</p>
      </div>
    );
}
```

CH 11 PERFORMANCE, ACCESSIBILITY & BEST PRACTICES

Performance Rules

- Use x, y, scale, rotation — GPU-accelerated, zero layout reflow
- Avoid width, height, top, left, margin — they trigger expensive layout recalculation
- Use gsap.quickTo() for mouse/cursor tracking (not regular gsap.to() per frame)
- Use overwrite: 'auto' to prevent conflicting tweens fighting each other
- Use ScrollTrigger.batch() for pages with dozens of scroll-triggered elements
- Set gsap.ticker.lagSmoothing(500, 33) to prevent jumps after returning to a tab

🇧🇩 বাংলা: সবচেয়ে গুরুত্বপূর্ণ: *x/y ব্যবহার করুন left/top নয়, quickTo() দিয়ে mouse track করুন, overwrite: 'auto'* দিন।

React Best Practices

- Always use useGSAP() — never useEffect() for GSAP code in React
- Always pass { scope: containerRef } to limit selector scope
- Always wrap event handlers with contextSafe() from useGSAP
- Register plugins ONCE at app top level — not inside component functions
- Store paused timelines in useRef, not useState
- Return cleanup functions from useGSAP for non-GSAP side effects (event listeners)

Accessibility

⬡ **Accessibility**

```
// Always respect prefers-reduced-motion!
useGSAP(() => {
  const mm = gsap.matchMedia();

  mm.add('(prefers-reduced-motion: no-preference)', () => {
    // Full animations for users OK with motion
    gsap.from('.hero', { y: 100, opacity: 0, duration: 1 });
  });

  mm.add('(prefers-reduced-motion: reduce)', () => {
    // Just show content — no animation at all
    gsap.set('.hero', { opacity: 1, y: 0 });
  });
}, { scope: container });
```

Common Mistakes

⚠ **WARNING** Using useEffect instead of useGSAP → Always use useGSAP() with @gsap/react

⚠ **WARNING** Forgetting gsap.registerPlugin(useGSAP) → Register at top of file or in main.jsx

⚠ **WARNING** No scope passed → Selectors leak into other components. Always pass { scope: ref }

⚠ **WARNING** Event handlers without contextSafe() → Animations won't be cleaned up on unmount

⚠ **WARNING** Registering plugins inside the component → Register ONCE outside / above the component

Debugging Tips

⬡ **Debugging**

```
// 1. ScrollTrigger markers (remove before deployment!)
scrollTrigger: { markers: true }

// 2. Slow-motion debug mode
gsap.globalTimeline.timeScale(0.1); // 10% speed

// 3. Inspect tween state
const t = gsap.to('.box', { x: 100 });
console.table({
  progress: t.progress(),
  time:     t.time(),
  duration: t.duration(),
  active:   t.isActive(),
  paused:   t.paused(),
});

// 4. See all active ScrollTriggers
ScrollTrigger.getAll().forEach(st =>
  console.log(st.vars.id || 'unnamed', '→ progress:', st.progress)
);

// 5. Kill everything and start fresh
gsap.killTweensOf('*');
ScrollTrigger.killAll();
```

React Component Template

⬡ **React Template**

```
// main.jsx — register ONCE
import gsap from 'gsap';
import { useGSAP } from '@gsap/react';
import { ScrollTrigger } from 'gsap/ScrollTrigger';
import { TextPlugin } from 'gsap/TextPlugin';
import { Flip } from 'gsap/Flip';
gsap.registerPlugin(useGSAP, ScrollTrigger, TextPlugin, Flip);

// ──────────────────────────────────────────────────────────
// YourComponent.jsx
import { useRef } from 'react';
import gsap from 'gsap';
import { useGSAP } from '@gsap/react';

export default function Component() {
  const container = useRef();
  const tlRef     = useRef(null);

  const { contextSafe } = useGSAP(() => {
    // Entrance animation
    gsap.from('.item', { opacity: 0, y: 40, stagger: 0.1, duration: 0.8 });

    // Paused timeline stored in ref
    tlRef.current = gsap.timeline({ paused: true })
      .to('.menu', { x: 0, duration: 0.5 });

    // Cleanup non-GSAP things
    return () => { /* removeEventListener, etc. */ };
  }, { scope: container, dependencies: [] });

  const openMenu  = contextSafe(() => tlRef.current?.play());
  const closeMenu = contextSafe(() => tlRef.current?.reverse());

  return (
    <div ref={container}>
      <button onClick={openMenu}>Open</button>
      <button onClick={closeMenu}>Close</button>
      <div className="menu" style={{ transform: 'translateX(-300px)' }}>
        <div className="item">Link 1</div>
        <div className="item">Link 2</div>
      </div>
    </div>
  );
}
```

Core Methods

Method / Property	Description / Example
`gsap.to(target, vars)`	Animate TO specified values from current state
`gsap.from(target, vars)`	Animate FROM specified values to current state
`gsap.fromTo(t, fromVars, toVars)`	Explicit start AND end — full control
`gsap.set(target, vars)`	Instant set — no animation
`gsap.timeline(vars)`	Create animation sequencer
`gsap.registerPlugin(...)`	Register plugins before using them
`gsap.killTweensOf(target)`	Kill all animations on a target
`gsap.getById('id')`	Get a tween by its id string
`gsap.matchMedia()`	Responsive animations per media query

Top Eases

Method / Property	Description / Example
`'power2.out'`	⭐ Most used — natural deceleration
`'power3.out'`	Snappier deceleration
`'back.out(1.7)'`	Bouncy overshoot at end
`'elastic.out'`	Spring / rubber-band feel
`'bounce.out'`	Ball bouncing at end
`'expo.out'`	Very sharp then very slow
`'sine.inOut'`	Smooth sine wave both ends
`'none'`	Linear (no easing)
`'steps(8)'`	8 discrete jumps

ScrollTrigger Quick Reference

Method / Property	Description / Example
`trigger`	Element that triggers animation
`start: 'top 80%'`	Trigger top hits 80% from top of viewport
`end: '+=500'`	End 500px after start position
`scrub: 1`	Link to scroll with 1s smoothing
`pin: true`	Stick element while scrolling through
`toggleActions: '...'`	'play none none reverse' — 4 string actions
`markers: true`	Debug markers — REMOVE before deploying!

Method / Property	Description / Example
`onUpdate: (self)=>{}`	self.progress (0-1), self.direction (1 or -1)

🎉

You Are Now a GSAP Expert!

আপনি এখন **GSAP Expert!** অভিনন্দন! 🇧🇩

Build something amazing. Start with gsap.to(). Add a timeline.
Then ScrollTrigger. Then the plugins. Then take over the web.

gsap.com/docs **gsap.com/get-started** **gsap.com/community**

GSAP Masterclass · useGSAP Hook · HTML + React · বাংলা ব্যাখ্যা